

LAB 2 · 카운터 : VS Code → Vivado

작성일 2026. 09. 11.

소스 작성 → 시뮬레이션 → 비트스트림 생성

첫 카운터 · Vivado 2026.1 실습

목차

프로젝트 시작

RTL·테스트벤치·XDC 작성

실행과 파형 확인

Vivado 구현과 bit 생성

실험 전 레포트

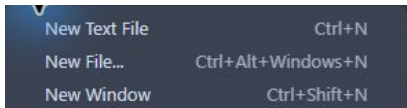
실험 후 레포트

v2.0.1 템플릿을 새 이름으로 clone

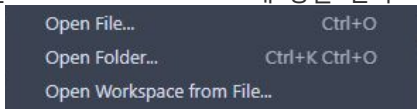
Windows PowerShell에서 한 줄씩 실행한다.

```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_01_counter
cd lab2_01_counter
git switch -c main
code LAB1.code-workspace
```

File → New Window



Ctrl+Shift+N 또는 File → New Window. 새 창을 연다.



File → Open Workspace from File...을 누른다.

workspace와 추천 확장

1. lab2_01_counter 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/counter4.v
src	src/input_frontend.v
src	src/lab2_counter.v
sim	sim/tb_counter4.sv
constraints	constraints/lab2_counter.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

`testbench`: `sim/tb_counter4.sv`

`simulation_top`: `tb_counter4`

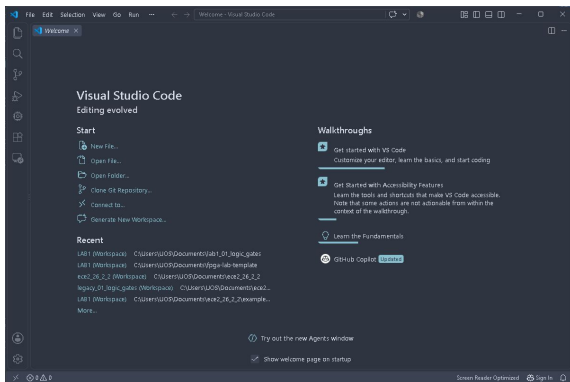
큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-9행

simulation.json를 열고 해당 구간을 직접 입력한다.

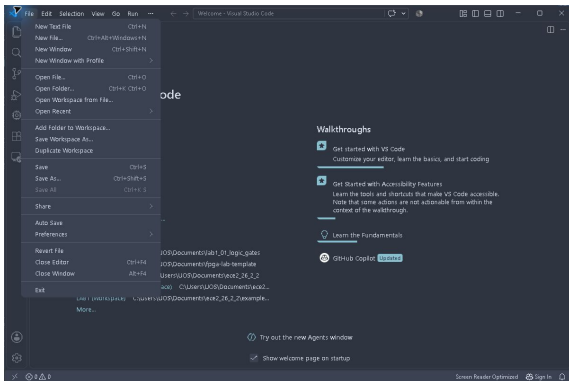
```
1  {  
2    "sources": [  
3      "src/counter4.v",  
4      "src/input_frontend.v",  
5      "src/lab2_counter.v"  
6    ],  
7    "testbench": "sim/tb_counter4.sv",  
8    "simulation_top": "tb_counter4"  
9  }
```

새 VS Code 창



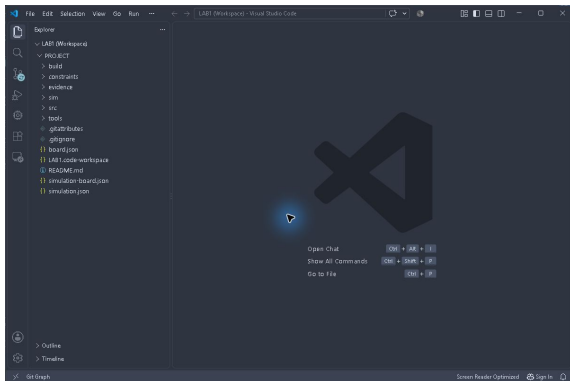
File → New Window 또는 Ctrl+Shift+N으로 새 창을 연다.

워크스페이스 열기



File → Open Workspace from File...에서 프로젝트의 LAB1.code-workspace를 고른다.

프로젝트 폴더 확인



Explorer에서 src.sim.constraints 폴더를 확인한다. 아래 캡처는 완성 예시 폴더에서 촬영했다.

카운터의 동작

rst=1이면 0으로 초기화한다. enable=0이면 현재 값을 유지한다.

enable=1에서 down=0이면 증가, down=1이면 감소한다.

4비트 값이므로 15 다음은 0, 0에서 감소하면 15가 된다.

counter4.v · 1-13행

src/counter4.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | module counter4 (
3 |     input wire clk, rst, enable, down,
4 |     output reg [3:0] value
5 | );
6 |     always @(posedge clk) begin
7 |         if (rst) value <= 4'd0;
8 |         else if (enable) begin
9 |             if (down) value <= value - 4'd1;
10 |            else value <= value + 4'd1;
11 |         end
12 |     end
13 | endmodule
```

input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17 assign reset = reset_pipe[1];
18 assign switches = sw_sync;
19 // Asynchronous assertion; release follows two main-clock edges.
20 always @(posedge clk or posedge rst) begin
21     if (rst) reset_pipe <= 2'b11;
22     else reset_pipe <= {reset_pipe[0], 1'b0};
23 end
24 always @(posedge clk or posedge reset) begin
25     if (reset) begin
26         button_meta <= 0; button_sync <= 0;
27         sw_meta <= 0; sw_sync <= 0;
28     end else begin
29         button_meta <= button; button_sync <= button_meta;
30         sw_meta <= sw; sw_sync <= sw_meta;
31     end
32 end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

lab2_counter.v · 1-16행

src/lab2_counter.v를 열고 해당 구간을 직접 입력한다.

```
1 | timescale 1ns/1ps
2 | // Combo II-DLD S75. Main clock B6 MUST be set to 1 kHz.
3 | // K4: reset. N8: step button. DIPSW1..8 = sw[7]..sw[0].
4 | module lab2_counter #(parameter integer STABLE_CYCLES = 20) (
5 |     input wire clk, rst, button,
6 |     input wire [7:0] sw,
7 |     output wire [7:0] led
8 | );
9 |     wire reset, press;
10 |    wire [7:0] switches;
11 |    input_frontend #(.STABLE_CYCLES(STABLE_CYCLES)) inputs(
12 |        clk, rst, button, sw, reset, press, switches);
13 |    wire [3:0] value;
14 |    counter4 core(clk, reset, press, switches[0], value);
15 |    assign led = {4'b0000, value};
16 | endmodule
```

tb_counter4.sv · 1-16행

sim/tb_counter4.sv를 열고 해당 구간을 직접 입력한다.

```
1  `timescale 1ns/1ps
2  module tb_counter4;
3  reg clk=0, rst=1;
4  integer checks=0;
5  always #5 clk=~clk;
6  task step; begin @(posedge clk); #1; end endtask
7  task check(input condition, input [8*100-1:0] description);
8  begin
9      checks=checks+1;
10     if (condition !== 1'b1) begin
11         $display("LAB2_FAIL %0s time=%0t",description,$time);
12         $fatal(1,"check failed");
13     end
14 end
15 endtask
16 task finish;
```

tb_counter4.sv · 17-32행

sim/tb_counter4.sv를 열고 해당 구간을 직접 입력한다.

```
17 begin
18     $display("LAB2_PASS counter4 checks=%0d",checks);
19     $finish;
20 end
21 endtask
22 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_counter4); end
23 initial begin #100000; $fatal(1,"watchdog timeout"); end
24 reg enable=0, down=0;
25 wire [3:0] value;
26 counter4 dut(clk,rst,enable,down,value);
27 integer i;
28 initial begin
29     step; check(value===0,"reset"); rst=0;
30     enable=1;
31     for(i=1;i<=16;i=i+1) begin
32         step; check(value==(i%16),"up including 15 to 0");
```

tb_counter4.sv · 33-43행

sim/tb_counter4.sv를 열고 해당 구간을 직접 입력한다.

```
33     end
34     down=1;
35     for(i=15;i>=0;i=i-1) begin
36         step; check(value===i,"down including 0 to 15");
37     end
38     enable=0; step; check(value===0,"hold");
39     enable=1; step; check(value===15,"down wrap");
40     rst=1; step; check(value===0,"reset beats enable");
41     finish;
42 end
43 endmodule
```

XDC 작성과 검증 범위

다음 세 화면의 XDC를 constraints/lab2_counter.xdc에 작성한다.

포트 이름을 lab2_counter.v와 대조한다.

Icarus는 XDC를 읽지 않는다. 이번 PASS는 counter4 코어의 기능 검사이며 보드 배선·핀·타이밍 검증은 별도 단계다.

lab2_counter.xdc • 1-16행

constraints/lab2_counter.xdc를 열고 해당 구간을 직접 입력한다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
```

lab2_counter.xdc • 17-32행

constraints/lab2_counter.xdc를 열고 해당 구간을 직접 입력한다.

```
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

lab2_counter.xdc • 33-42행

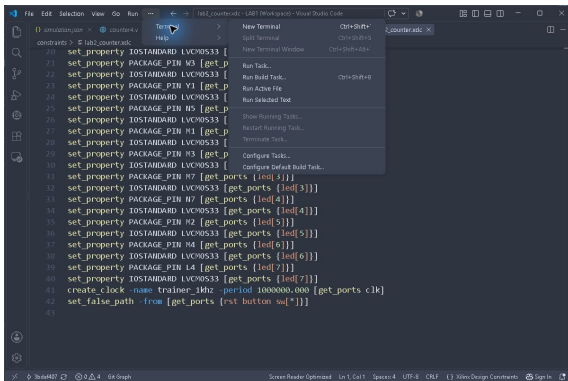
constraints/lab2_counter.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property PACKAGE_PIN N7 [get_ports {led[4]]}
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]]}
35 set_property PACKAGE_PIN M2 [get_ports {led[5]]}
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]]}
37 set_property PACKAGE_PIN M4 [get_ports {led[6]]}
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]]}
39 set_property PACKAGE_PIN L4 [get_ports {led[7]]}
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]]}
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]]}]
```

VS Code에서 실행

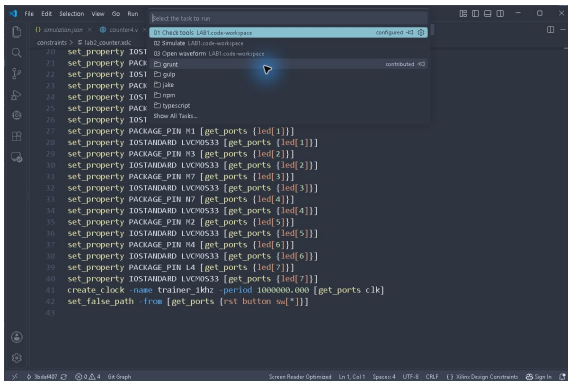
1. File → Save All.
2. 상단 Terminal → Run Task...를 누른다. 메뉴가 접혀 있으면 ...를 누른다.
3. 01 Check tools로 도구 경로를 확인한 뒤 02 Simulate를 선택한다.

Run Task 메뉴



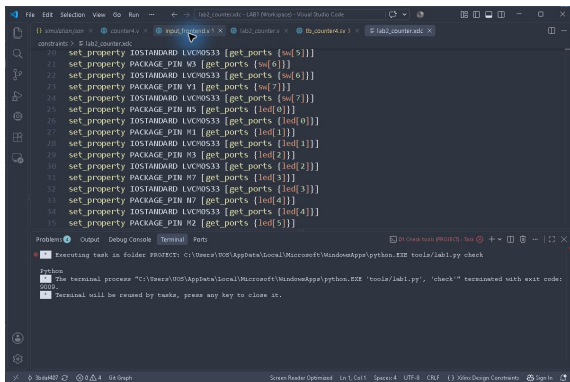
Terminal → Run Task...를 누른다.

시뮬레이션 작업 선택



01 Check tools: 설치 확인. 02 Simulate: 컴파일과 테스트벤치 실행.

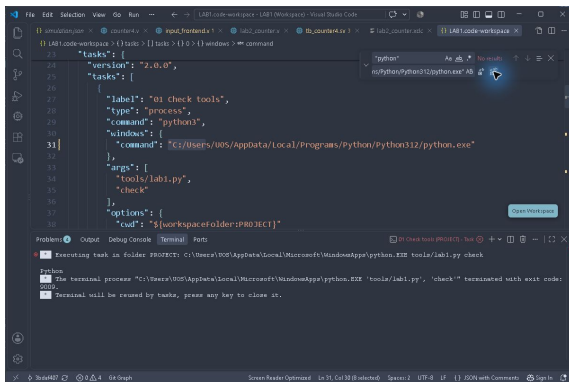
Python 실행 오류를 만났다면



The screenshot shows a Visual Studio Code window with a Python script in the editor and a terminal window at the bottom. The script contains 35 lines of code, each setting a property for a device (IOSTANDARD or PACKAGE_PIN) to a specific GPIO pin. The terminal window shows an error message: "Executing task in folder PROJECT: C:\Users\9009\AppData\Local\Microsoft\WindowsApps\python.EXE tools\lab1.py check". Below this, it says "Python" and "The terminal process 'C:\Users\9009\AppData\Local\Microsoft\WindowsApps\python.EXE 'tools\lab1.py', 'check'' terminated with exit code: 9009." The error code 9009 is highlighted in red.

9009와 WindowsApps 경로가 나오면 Python 실행 별칭이 선택된 것이다.

설치된 Python 경로 지정



```
23  "tasks": {
24    "version": "2.0.0",
25    "tasks": [
26      {
27        "label": "01 Check tools",
28        "type": "process",
29        "command": "python3",
30        "windows": {
31          "command": "C:/Users/UOS/AppData/Local/Programs/Python/Python312/python.exe"
32        },
33        "args": [
34          "tools/lab1.py",
35          "check"
36        ],
37        "options": {
38          "cwd": "${workspacefolder:PROJECT}"
39        }
40      }
41    ]
42  }
```

Output

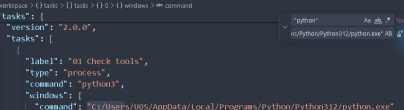
```
01 Check tools [PROJECT]: No output
Python
The terminal process "C:/Users/UOS/AppData/Local/Programs/Python/Python312/python.exe" "tools/lab1.py", "check" terminated with exit code: 2009.
Terminal will be reused by tasks, press any key to close it.
```

LAB1.code-workspace의 Windows command를 자신의 실제 python.exe 경로로 바꾸고 저장한다. 캡처의 UOS 경로는 이 PC의 예시다.

실행 경로를 수정할 때

1. Ctrl+P → LAB1.code-workspace를 연다.
2. tasks 안의 windows → command 항목을 찾는다.
3. 세 작업에 있는 Python 실행 경로를 자신의 설치 경로로 수정한다.
4. JSON 경로 구분자는 /를 사용한다. Ctrl+S로 저장한다.
5. Terminal → Run Task... → 02 Simulate를 다시 실행한다.

실제 실행 결과



```
23  "tasks": {
24    "version": "2.0.0",
25    "tasks": [
26      {
27        "label": "01 check tools",
28        "type": "process",
29        "command": "python3",
30        "windows": [
31          {
32            "command": "C:/Users/005/AppData/Local/Programs/Python/Python312/python.exe"
33          }
34        ],
35        "args": [
36          "tools/lab1.py",
37          "check"
38        ],
39      }
40    ],
41    "options": {
42      "cwd": "${workspaceFolder:PROJECT}"
43    }
44  }
```

LAB2_PASS counter4 checks=36. 종료 시각 356000 ps = 356 ns.
2026-09-11 GUI 실행에서 확인했다.

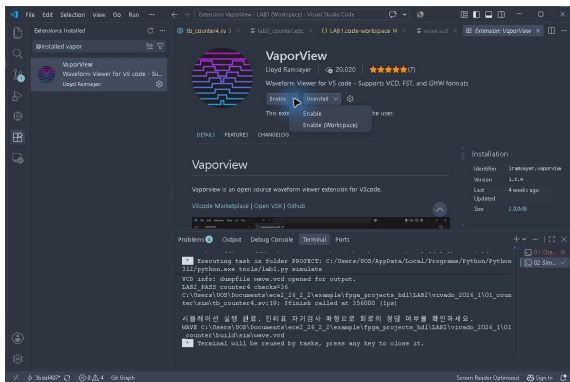
PASS가 확인한 내용

리셋, 증가와 15→0 순환, 감소와 0→15 순환, enable=0 유지, 리셋 우선순위를 총 36회 검사한다.

이번 테스트벤치의 DUT는 counter4이다. 입력 동기화·디바운스와 실제 핀 동작까지 통과했다는 의미는 아니다.

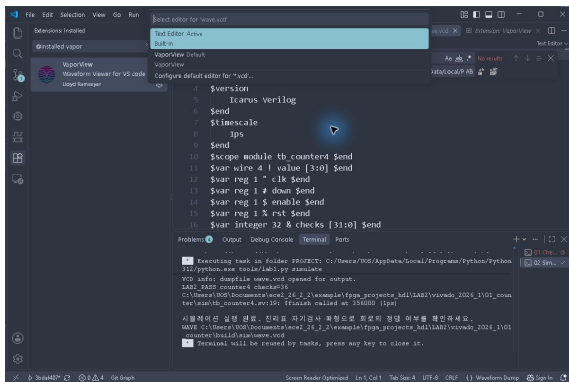
편집기의 경고 4개는 남아 있다. 경고 0개로 기록하지 않는다.

VaporView가 비활성화되어 있다면



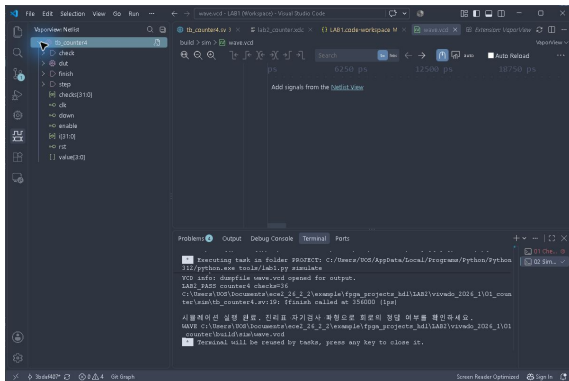
Extensions에서 VaporView를 찾고 Enable 옆 화살표 → Enable (Workspace)를 선택한다.

VCD를 파형으로 열기



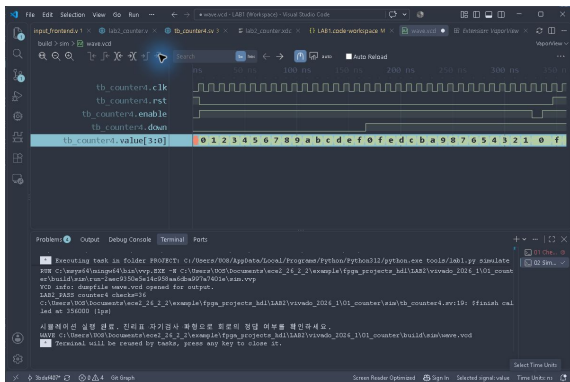
build/sim/wave.vcd를 연다. 텍스트로 보이면 탭 우클릭 → Reopen Editor With... → VaporView.

관찰할 신호 추가



Netlist View에서 tb_counter4를 펼친다. clk·rst·enable·down·value[3:0]를 더블클릭해 추가한다.

전체 파형 확인



Zoom Fit으로 전체 시간을 맞춘다. 신호 이름 열을 넓히고 하단 시간 단위를 ns로 바꾼다.

파형을 읽는 순서

1. clk 상승 에지 직후 value가 바뀌는지 본다.
2. down=0의 증가 구간과 down=1의 감소 구간을 비교한다.
3. enable=0 구간에서 value가 유지되는지 확대한다.
4. 마지막 rst=1 구간과 356 ns 종료를 확대해 확인한다.
5. 신호 이름·시간축·입력·출력이 함께 보이게 캡처한다.

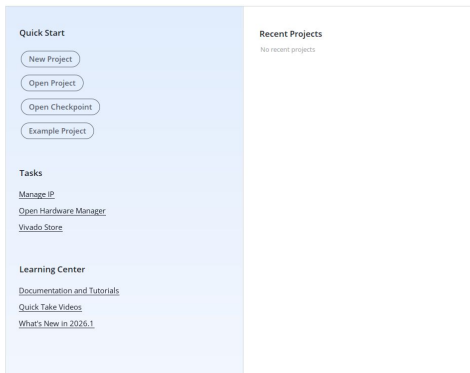
실험 전 레포트

1. RTL과 테스트벤치가 맡은 역할을 설명한다.
2. 초기화·증가·감소·순환·유지의 예상 결과를 표로 적는다.
3. 자신이 실행한 PASS 로그와 파형을 넣고 각 구간을 설명한다.
4. Python 경로 등 자신이 고친 설정과 발생한 오류를 적는다.
5. XDC는 작성했지만 이번 기능 시뮬레이션에 사용되지 않았음을 구분한다.

Vivado에서 이어서 구현

1. VS Code에서 작성한 RTL·TB·XDC를 모두 저장한다.
2. Vivado 2026.1을 실행한다.
3. 새 RTL 프로젝트에 작성한 파일을 등록한다.
4. 같은 테스트벤치로 시뮬레이션한 뒤 bit를 만든다.

New Project



Quick Start → New Project를 누른다.

프로젝트 이름과 위치

Project name: ×

Project location: × ...

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/lab2_01_counter/vivado

이름은 lab2_counter. 자신의 실습 폴더 아래 vivado 폴더를 지정한다.
Create project subdirectory 해제 → Next.

RTL Project 선택

- ☒ RTL Project
You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL analysis, synthesis, implementation, design planning and analysis.
- ☒ Do not specify sources at this time
- ☐ Project is an extensible Vitis platform

RTL Project와 Do not specify sources at this time을 선택한 채 Next.
파일은 프로젝트 생성 후 직접 등록한다.

정확한 FPGA 부품 선택

Parts

Boards

Q

xc7s75fgga484-1

(3 matches)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops	Block RAMs	Ultra RAMs	DSPs	BUFGs	Gb Transceivers	GTPE
xc7s75fgga484-1	484	338	48000	96000	90	0	140	32	0	0
xc7s75fgga484-1IL	484	338	48000	96000	90	0	140	32	0	0
xc7s75fgga484-1Q	484	338	48000	96000	90	0	140	32	0	0

Parts 검색란에 xc7s75fgga484-1 입력 → 정확히 -1로 끝나는 행 선택 → Next. -1IL, -1Q와 구분한다.

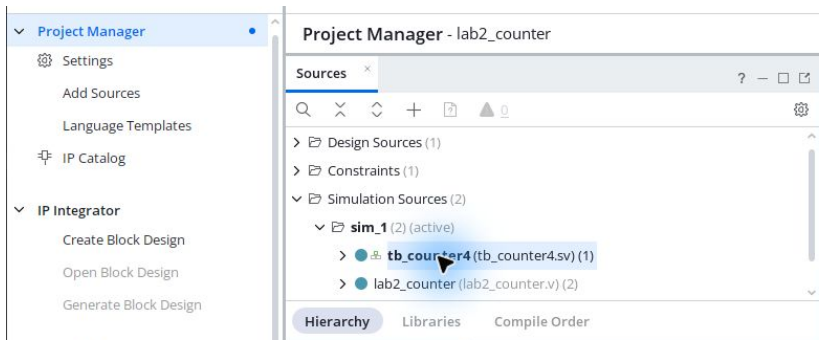
생성 전 마지막 확인

New Project Summary

- A new RTL project named 'lab2_counter' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

Spartan-7, fgga484, Speed Grade -1을 확인하고 Finish.

Add Sources



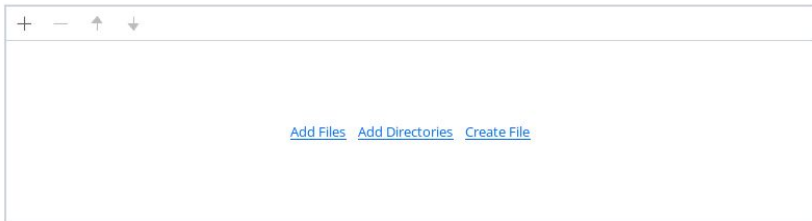
왼쪽 Project Manager → Add Sources를 누른다. 아래 화면은 파일 등록을 마친 예시다.

RTL은 Design Sources

- ☐ Add or greate constraints
- ☒ Add or create design sources
- ☐ Add or create simulation sources

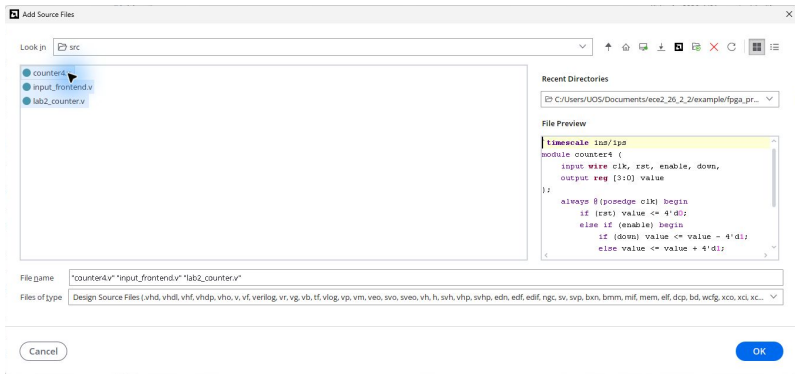
Add or create design sources 선택 → Next.

Add Files 선택



Add Files를 누르고 VS Code 프로젝트의 src 폴더로 이동한다.

RTL 세 파일 선택



counter4.v · input_frontend.v · lab2_counter.v를 함께 선택하고 OK.

원본 RTL을 참조하도록 등록

	Index	Name	Library	Version	Location
●	1	counter4.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/example
●	2	input_frontend.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/example
●	3	lab2_counter.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/example

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories

세 파일 이름 확인 → Copy sources into project는 해제 → Finish. 이후 VS Code 수정 내용은 저장 후 Vivado에 반영한다.

TB는 Simulation Sources

- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒ Add or create simulation sources

Add Sources → Add or create simulation sources → Next → Add Files.

테스트벤치 등록

Specify simulation set: sim_1

+ - ↑ ↓					
	Index	Name	Library	Version	Location
	1	tb_counter4.sv	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/example/f

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

sim/tb_counter4.sv 등록 → Include all design sources for simulation
유지 → Finish.

XDC는 Constraints

- ☒ Add or create constraints
- ☐ Add or create design sources
- ☐ Add or create simulation sources

Add Sources → Add or create constraints → Next → Add Files.

핀 제약 파일 등록

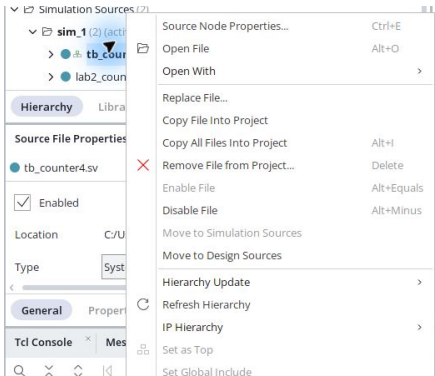
Specify constraint set: constrs_1 (active) ▼

Constraint File	Location
lab2_counter.xdc	C:\Users\UOS\Documents\ece2_26_2\example\fpga_projects_hdl\LAB2\vivado_2026_1\01_counter\constra

☐ Copy constraints files into project

constraints/lab2_counter.xdc를 등록한다. Copy constraints files into project 해제 → Finish.

시뮬레이션 최상위 지정



Simulation Sources → sim_1 → tb_counter4 우클릭 → Set as Top. 이미
짧은 이름이면 지정된 상태이며 메뉴가 비활성화된다.

두 최상위 모듈 구분

The screenshot displays the Vivado IDE interface. On the left, the 'Sources' pane shows a project hierarchy with 'Simulation Sources (2)' expanded, listing 'sim_1 (2) (active)' and 'lab2_counter (lab2_counter.v) (2)'. Below this, the 'Source File Properties' for 'tb_counter4.sv' are shown, indicating it is 'Enabled' and its 'Type' is 'SystemVerilog'. On the right, the 'Project Summary' pane for 'tb_counter4.sv' is visible. The 'Overview' tab shows project settings: Project name 'lab2_counter', Project location 'C:/Users/U05/Documents/ece2_26_2_2/tmp/lab2_counter_gui', Product family 'Spartan-7', Project part 'xc7s75fpga484-1', Top module name 'lab2_counter', Target language 'Verilog', Simulator language 'Mixed', and Target Simulator 'Vivado Simulator'. Below this, the 'Synthesis' and 'Implementation' sections both show a status of 'Complete' with 'No errors'.

Synthesis		Implementation	
Status	✓ Complete	Status	✓ Complete
Messages	⚠ 12 warnings	Messages	No errors
Part	xc7s75fpga484-1	Part	xc7s75fpga484-1

Simulation Sources의 굵은 이름은 tb_counter4. 오른쪽 Project Summary의 Top module name은 lab2_counter.

Behavioral Simulation 실행

The screenshot shows the IP Integrator interface. On the left, the 'Simulation' menu is expanded, showing 'Run Simulation' and a sub-menu with options like 'Run Behavioral Simulation'. In the center, a table lists components: 'dut' (counter4, Verilog IV) and 'glbl' (glbl, Verilog IV). On the right, a table lists signals: 'rst' (1, Logic), 'checks[31:0]' (36, Array), 'enable' (1, Logic), 'down' (1, Logic), 'value[3:0]' (0, Array), and 'i[31:0]' (-1, Array).

Component	Instance Name	Language
dut	counter4	Verilog IV
glbl	glbl	Verilog IV

Signal	Value	Type
rst	1	Logic
checks[31:0]	36	Array
enable	1	Logic
down	1	Logic
value[3:0]	0	Array
i[31:0]	-1	Array

왼쪽 Run Simulation → Run Behavioral Simulation을 누른다.

36개 검사 완료

The screenshot displays a Verilog simulation environment. The **Scope** window on the left shows a tree structure with `tb_counter4` containing `dut` and `gbl`. The **Objects** window shows variables like `clk`, `rst`, `checks[310]`, `enable`, `down`, `value[30]`, and `z[310]` with their respective values and data types. The main editor shows Verilog code for `tb_counter4`, including initialization, a loop for checking values, and a `$finish` statement. The **Tcl Console** at the bottom shows the command `run` and the output `run output will be captured here: C:/Users/V08/Documents/ece2_24_2/tmp/lab2_counter_gul/lab2_counter.run/smp1_1/runme.log`. The status bar at the bottom right indicates **Sim Time: 356 ns**.

checks=36, Sim Time=356 ns. VS Code 실행 결과와 비교한다. \$finish 위치에서 멈추는 것은 정상이다.

전체 파형 보기



파형 탭을 선택하고 우상단 사각형으로 확대한다. Zoom Fit을 눌러 0~356 ns 전체 구간을 확인한다.

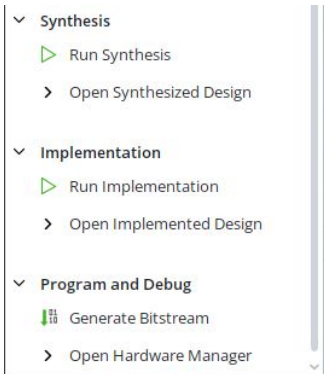
파형과 시뮬레이션 로그

GUI 실행 로그에도 LAB2_PASS counter4 checks=36과 356 ns 종료가 남는다.

value는 16진수로 표시된다. a~f는 10~15다. 증가·감소·유지·초기화 구간을 VS Code 파형과 대조한다.

코어 테스트벤치의 통과와 실제 보드 버튼·LED 동작 확인은 별개다.

Generate Bitstream



왼쪽 Generate Bitstream을 누른다. 합성·구현 실행 여부를 물으면 Yes.
Launch runs on local host 선택 → OK.

합성 → 구현 → bit 생성

1. 합성은 RTL을 FPGA 논리 소자로 변환한다.
2. 구현은 소자를 배치하고 신호를 배선한다.
3. 마지막으로 FPGA에 기록할 .bit 파일을 생성한다.
4. 우상단 진행 상태와 하단 Design Runs를 확인하며 기다린다.
5. 실패하면 Messages의 첫 Error를 확인한다. 완료 전 같은 작업을 다시 실행하지 않는다.

비트스트림 생성 완료

Tcl Console Messages Log Reports Design Runs ? - □ ↻															
Q × ↕ ⏮ ⏪ ⏩ ⏭ + %															
Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF
✓ synth_1	constrs_1	synth_design Complete!												10	31
✓ impl_1	constrs_1	write_bitstream Complete!	99999'	0.000	0.122	0.000		0.000	0.103	0	8 Warn			11	17

Design Runs의 impl_1 상태가 write_bitstream Complete! 인지 확인한다.
이번 GUI 실행은 2026-09-11에 완료했다.

생성된 파일 찾기

프로젝트 폴더 → lab2_counter.runs → impl_1 → lab2_counter.bit

이 파일이 FPGA에 기록할 비트스트림이다. .xpr는 Vivado 프로젝트 설정 파일이다.

파일 탐색기에서 수정 시각을 확인한다. 소스를 수정했다면 합성·구현·bit 생성까지 다시 실행해야 한다.

이번 구현의 경고 해석

상위 LED 4개는 코드에서 0으로 고정하므로 상수 출력 경고가 발생한다.

TIMING-18 경고는 외부 입출력 지연 제약이 없는 항목이다. 버튼은 비동기 입력이고 LED는 외부 클록으로 샘플링되는 인터페이스가 아니다.

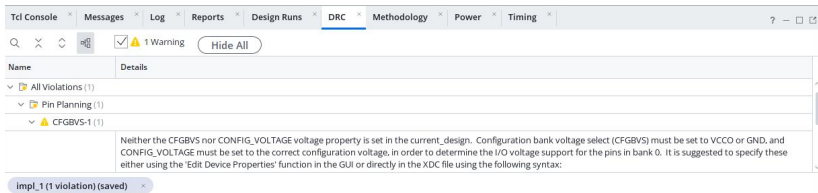
1 kHz 클록의 내부 경로 타이밍과 DRC를 확인한다. 이 결과를 모든 외부 인터페이스의 타이밍 검증 완료로 해석하지 않는다.

구현 후 타이밍 확인

Setup			Hold			Pulse Width		
Worst Negative Slack (WNS)	999997.000 ns		Worst Hold Slack (WHS)	0.122 ns		Worst Pulse Width Slack (WPWS)	499999.375 ns	
Total Negative Slack (TNS)	0.000 ns		Total Hold Slack (THS)	0.000 ns		Total Pulse Width Negative Slack (TPWS)	0.000 ns	
Number of Failing Endpoints	0		Number of Failing Endpoints	0		Number of Failing Endpoints	0	
Total Number of Endpoints	33		Total Number of Endpoints	33		Total Number of Endpoints	18	

Open Implemented Design → Timing. Setup·Hold의 Failing Endpoints가 0 인지 확인한다. 현재 WHS는 0.122 ns다.

DRC 결과 읽기



DRC 탭을 연다. 이번 결과는 오류 0개, CFGBVS-1 경고 1개다. 구성 बैं크 전압 속성 미지정 경고이며 bit 생성은 완료됐다.

CFGBVS 경고를 수정하려면

CFGBVS와 CONFIG_VOLTAGE는 구성 बैं크의 실제 보드 배선·전압에 맞춰 지정하는 속성이다.

일반 I/O의 LVCMOS33 설정만 보고 구성 बैं크 전압도 같다고 추정하지 않는다. 보드 회로도에서 확인한 뒤 XDC에 반영한다.

이번 제작 검증에서는 이 속성을 임의로 추가하지 않았다. 레포트에는 남은 경고와 확인 범위를 함께 적는다.

LED 핀 확인

Tcl ConsoleMessagesLogReportsDesign RunsI/O PortsDRCMethodologyPower

Window → I/O Ports → 패널 확대 → Expand All. led[0..7]의 Package Pin
과 LVC MOS33을 XDC와 대조한다.

스위치·버튼·클록 핀 확인

▼ sw (8)	IN			☒	34	LVC MOS33*	▼	3.300
sw[7]	IN		Y1	▼ ☒	34	LVC MOS33*	▼	3.300
sw[6]	IN		W3	▼ ☒	34	LVC MOS33*	▼	3.300
sw[5]	IN		U2	▼ ☒	34	LVC MOS33*	▼	3.300
sw[4]	IN		T1	▼ ☒	34	LVC MOS33*	▼	3.300
sw[3]	IN		W4	▼ ☒	34	LVC MOS33*	▼	3.300
sw[2]	IN		W1	▼ ☒	34	LVC MOS33*	▼	3.300
sw[1]	IN		V4	▼ ☒	34	LVC MOS33*	▼	3.300
sw[0]	IN		U4	▼ ☒	34	LVC MOS33*	▼	3.300
▼ Scalar ports (3)								
button	IN		N8	▼ ☒	35	LVC MOS33*	▼	3.300
clk	IN		B6	▼ ☒	36	LVC MOS33*	▼	3.300
rst	IN		K4	▼ ☒	35	LVC MOS33*	▼	3.300

sw[0]=U4, button=N8, clk=B6, rst=K4. sw[0]은 보드 DIPSW8에 대응한다.
클록은 1 kHz로 설정한다.

보드에서 실행하는 순서

1. Combo II-DLD S75의 전원·JTAG 연결을 확인하고 메인 클록을 1 kHz로 맞춘다.
2. Vivado 왼쪽 Open Hardware Manager를 누른다.
3. Open Target → Auto Connect를 선택한다.
4. 검색된 xc7s75 장치를 우클릭 → Program Device.
5. Bitstream file에서 lab2_counter.bit 선택 → Program.

기록 후 동작 확인

1. 리셋 입력(K4)을 활성화해 하위 LED 4개가 0인지 확인한다.
2. 방향 입력 `sw[0]`(DIPSW8)을 0으로 두고 N8 버튼을 눌렀다 놓는다.
한 번에 1씩 증가해야 한다.
3. 15에서 한 번 더 누르면 0으로 순환하는지 본다.
4. `sw[0]=1`로 바꾸고 버튼을 눌러 감소와 0→15 순환을 확인한다.
5. 버튼을 길게 누르면 연속 증가하지 않고, 놓았다 다시 눌러야 다음 값으로 바뀌는지 확인한다.

실험 후 레포트

1. Vivado의 소스 계층·시뮬레이션 PASS·파형을 넣고 VS Code 결과와 비교한다.
2. 사용한 부품·핀·1 kHz 클록 제약과 합성·구현 결과를 기록한다.
3. write_bitstream Complete 화면, 생성 파일, 남은 경고와 해석을 첨부한다.
4. 실제 보드의 초기화·증가·감소·순환 동작을 사진과 영상으로 남긴다.
5. 자신의 GitHub 저장소에 소스·레포트·사진·영상 링크를 올리고 README에서 연결한다.

제작 검증 범위

완료: VS Code와 Vivado에서 동일 코어 테스트벤치 36개 검사 통과, 356 ns 종료.

완료: xc7s75fgga484-1 합성·배치·배선·비트스트림 생성, 내부 타이밍 및 핀 배치 확인.

이 자료의 보드 기록·사진·영상 단계는 학생 실험 절차다. 실제 장치 기록과 물리 동작은 이번 제작 검증에 포함되지 않았다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src.sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내